

INFORME DE LABORATORIO

Simulación de una Red Local con Kali Linux y Debian

Versión simplificada de la actividad INFRA-10

Alumno	Renzo Rampoldi
Fecha	06/09/2026
Entorno	Oracle VirtualBox
Máquinas virtuales	Kali Linux 2024.3 y Debian GNU/Linux 11

Objetivo: demostrar conectividad, servicios, escaneo, análisis de tráfico y filtrado mediante firewall en una red virtual aislada.

1. Introducción

La actividad original INFRA-10 propone construir una mini-red segmentada con una red interna, una DMZ, una WAN simulada y un firewall pfSense. **Para este laboratorio se realizó una adaptación simplificada** con el objetivo de practicar los mismos conceptos básicos de conectividad, exposición de servicios, descubrimiento de puertos, análisis de paquetes y control de tráfico, pero utilizando únicamente dos máquinas virtuales.

La simulación se mantuvo completamente aislada de la red física del equipo anfitrión mediante una “Red interna” de VirtualBox denominada **labLAN**. De esta forma se evitó utilizar el modo puente o la red real del equipo, respetando el enfoque de laboratorio controlado planteado por la consigna original.

2. Topología implementada

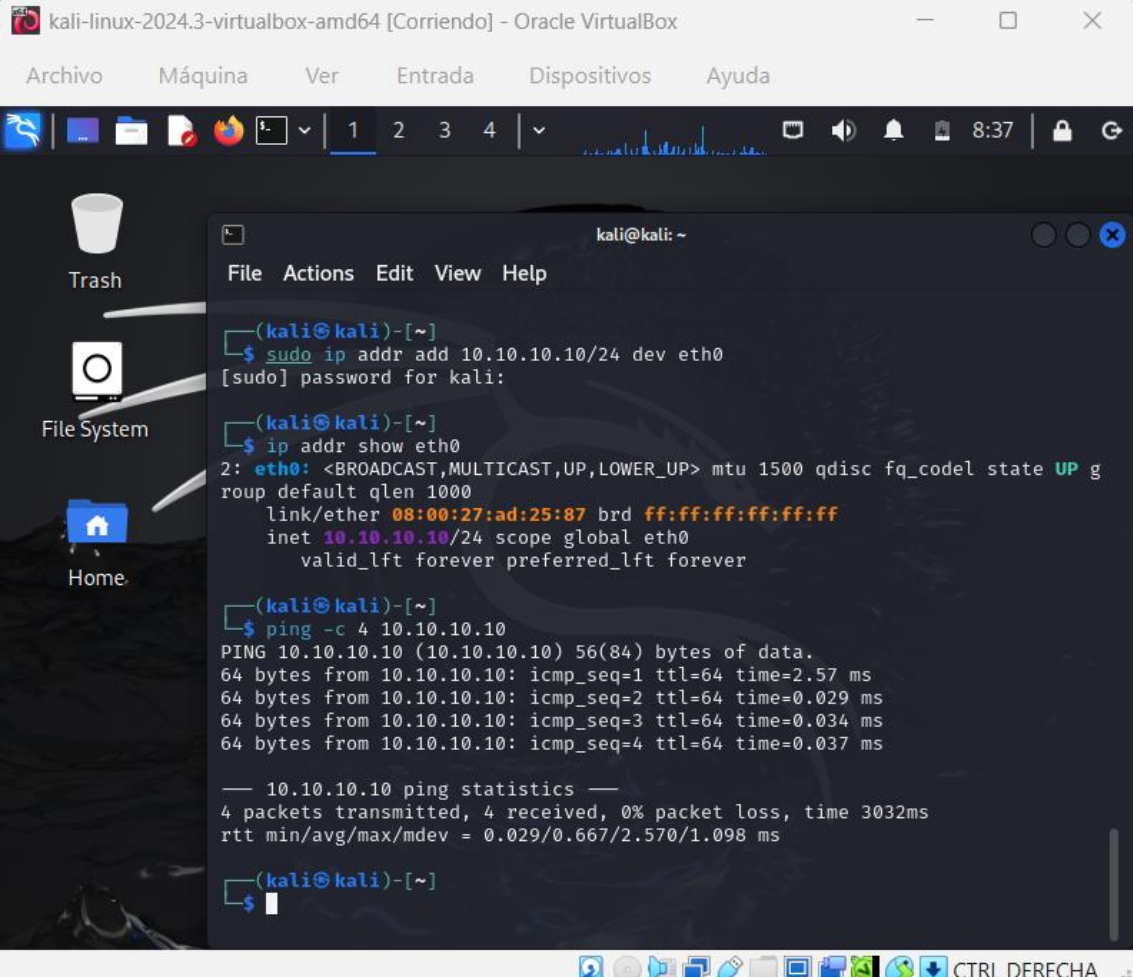
Kali Linux 10.10.10.10 ⇄ labLAN ⇄ Debian 10.10.10.20

Kali Linux se utilizó como cliente y estación de análisis. Debian se utilizó como servidor, alojando servicios SSH y HTTP mediante Apache.

Equipo	Función	Interfaz/IP	Servicios principales
Kali Linux	Cliente / análisis	eth0 – 10.10.10.10/24	Nmap, Wireshark, curl, ping
Debian 11	Servidor	enp0s3 – 10.10.10.20/24	SSH (22/tcp), Apache HTTP (80/tcp)

3. Configuración de direccionamiento

En Kali Linux se asignó manualmente la dirección 10.10.10.10/24 sobre la interfaz eth0. Debido a que la red interna de VirtualBox no posee DHCP, fue necesario configurar la dirección IPv4 manualmente.



The screenshot shows a Kali Linux terminal window titled "kali-linux-2024.3-virtualbox-amd64 [Corriendo] - Oracle VirtualBox". The terminal displays the following commands and output:

```
(kali@kali)-[~]
$ sudo ip addr add 10.10.10.10/24 dev eth0
[sudo] password for kali:

(kali@kali)-[~]
$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:ad:25:87 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.10/24 scope global eth0
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$ ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=2.57 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.029 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.034 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.037 ms

— 10.10.10.10 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3032ms
rtt min/avg/max/mdev = 0.029/0.0667/2.570/1.098 ms

(kali@kali)-[~]
$
```

Figura 1. Kali Linux con la dirección 10.10.10.10/24 configurada en eth0.

En Debian se configuró la dirección 10.10.10.20/24 sobre la interfaz enp0s3. Una vez aplicada la configuración, se comprobó conectividad tanto hacia la propia dirección de Debian como hacia Kali Linux.

```

Debian [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:fd:99:a3 brd ff:ff:ff:ff:ff:ff
renzo@Debian:~$ sudo ip addr add 10.10.10.20/24 dev enp0s3
renzo@Debian:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:fd:99:a3 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.20/24 scope global enp0s3
        valid_lft forever preferred_lft forever
renzo@Debian:~$ ping -c 4 10.10.10.20
PING 10.10.10.20 (10.10.10.20) 56(84) bytes of data.
64 bytes from 10.10.10.20: icmp_seq=1 ttl=64 time=0.248 ms
64 bytes from 10.10.10.20: icmp_seq=2 ttl=64 time=0.046 ms
64 bytes from 10.10.10.20: icmp_seq=3 ttl=64 time=0.044 ms
64 bytes from 10.10.10.20: icmp_seq=4 ttl=64 time=0.036 ms

--- 10.10.10.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3056ms
rtt min/avg/max/mdev = 0.036/0.093/0.248/0.089 ms
renzo@Debian:~$ ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=9.06 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.927 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.955 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.945 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.927/2.972/9.061/3.515 ms
renzo@Debian:~$ _

```

Figura 2. Debian con IP 10.10.10.20/24 y prueba de conectividad hacia Kali Linux.

4. Prueba de conectividad

La primera validación consistió en utilizar ICMP mediante el comando ping. La comunicación entre 10.10.10.20 y 10.10.10.10 presentó 0 % de pérdida de paquetes, demostrando que ambas máquinas pertenecían a la misma red virtual y podían comunicarse correctamente.

5. Servidor web en Debian

Se utilizó Apache HTTP Server en Debian para representar un servicio disponible dentro de la red local. El estado del servicio fue verificado mediante systemctl, obteniendo como resultado “active (running)”.

Desde Kali Linux se verificó la disponibilidad del servicio HTTP accediendo a la dirección 10.10.10.20. La respuesta HTTP 200 OK confirmó que el servidor web atendía correctamente las solicitudes por el puerto 80.

Informe técnico - 06/09/2026

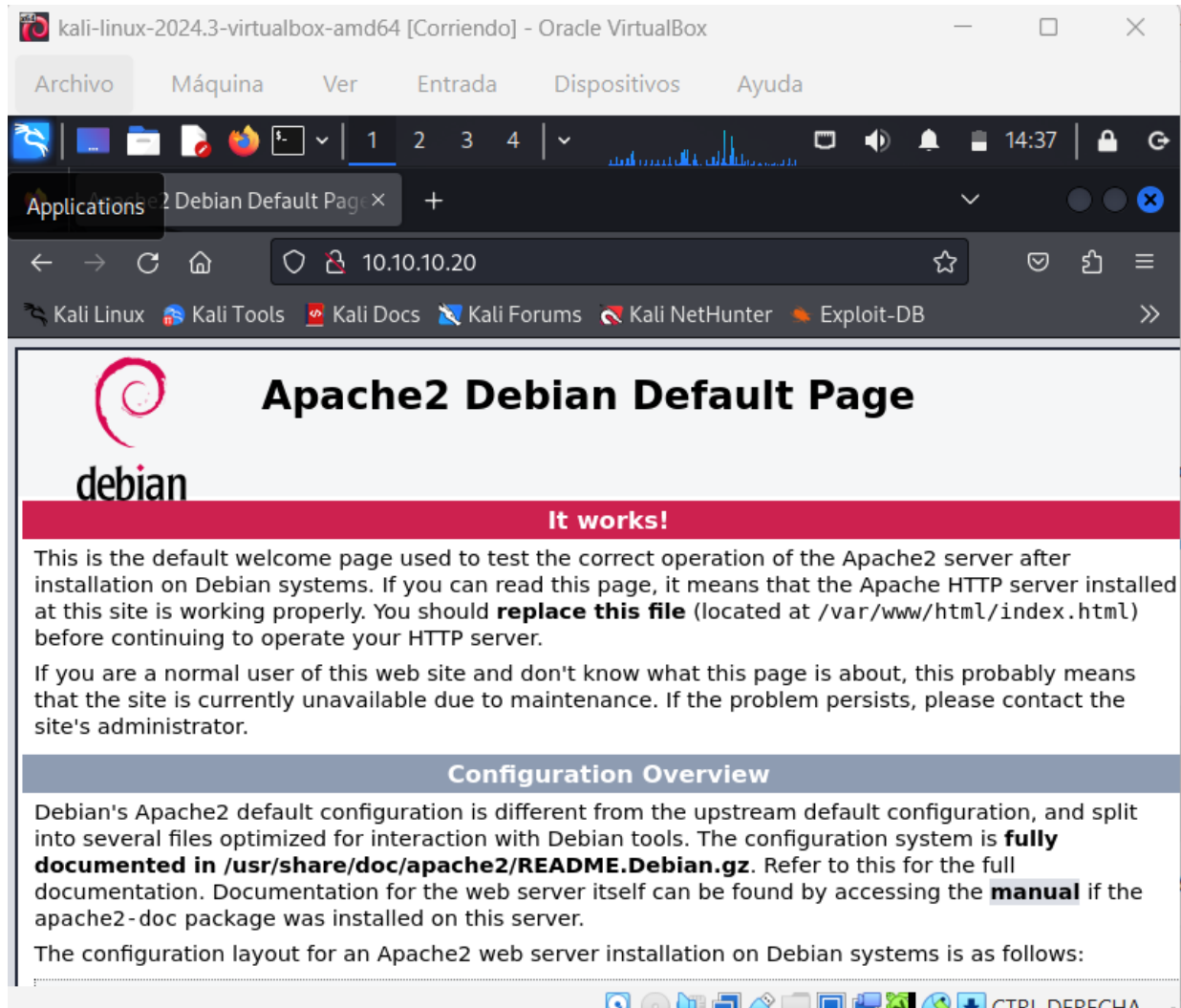
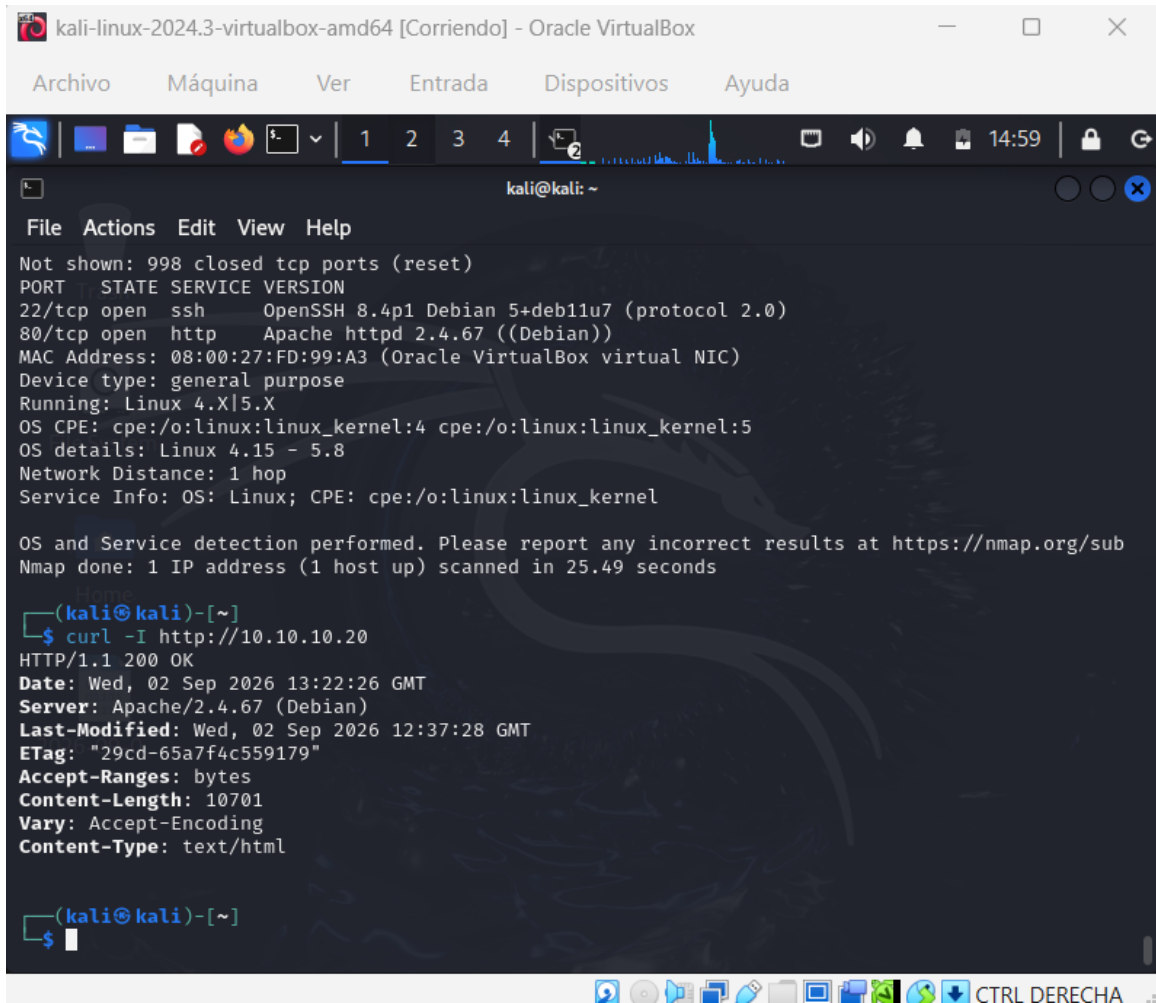


Figura 4. Acceso desde el navegador de Kali a la página predeterminada de Apache2 en Debian.



```

kali-linux-2024.3-virtualbox-amd64 [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda

kali@kali: ~
File Actions Edit View Help
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u7 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Debian))
MAC Address: 08:00:27:FD:99:A3 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/sub
Nmap done: 1 IP address (1 host up) scanned in 25.49 seconds

(kali@kali)-[~]
$ curl -I http://10.10.10.20
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 13:22:26 GMT
Server: Apache/2.4.67 (Debian)
Last-Modified: Wed, 02 Sep 2026 12:37:28 GMT
ETag: "29cd-65a7f4c559179"
Accept-Ranges: bytes
Content-Length: 10701
Vary: Accept-Encoding
Content-Type: text/html

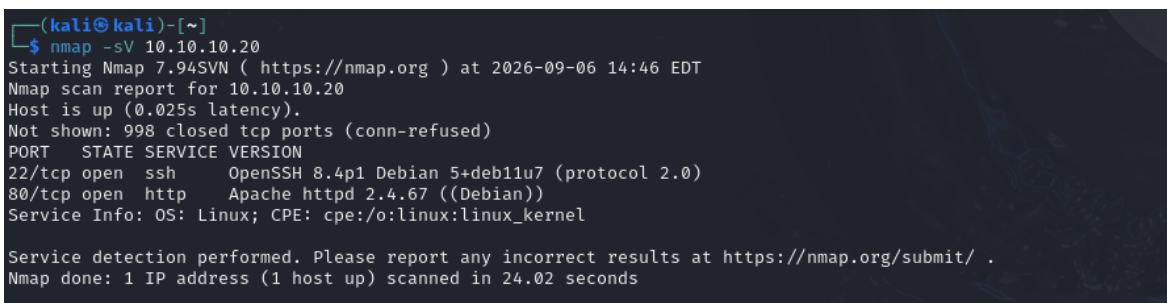
(kali@kali)-[~]
$

```

Figura 5. Respuesta HTTP 200 OK obtenida desde Kali antes de aplicar el firewall.

6. Escaneo y reconocimiento con Nmap

Se utilizó Nmap desde Kali Linux para descubrir los servicios expuestos por Debian. El escaneo de versiones identificó dos puertos abiertos: 22/tcp correspondiente a SSH y 80/tcp correspondiente a HTTP/Apache.



```

(kali@kali)-[~]
$ nmap -sV 10.10.10.20
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-06 14:46 EDT
Nmap scan report for 10.10.10.20
Host is up (0.025s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u7 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Debian))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 24.02 seconds

```

Figura 6. Escaneo Nmap -sV: detección de SSH y Apache en Debian.

Posteriormente se realizó un escaneo combinado con las opciones -sS, -sV y -O. Esta prueba permitió reunir en una sola salida el estado de los puertos, versiones de servicio y una estimación del sistema operativo remoto.

```

(kali@kali)-[~]
$ sudo nmap -sS -sV -O 10.10.10.20
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-06 14:51 EDT
Nmap scan report for 10.10.10.20
Host is up (0.0027s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u7 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Debian))
MAC Address: 08:00:27:FD:99:A3 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.49 seconds

(kali@kali)-[~]
$

```

Figura 7. Escaneo Nmap combinado con SYN scan, detección de versiones y estimación del sistema operativo.

Interpretación: Nmap detectó que el host 10.10.10.20 estaba activo, encontró 22/tcp y 80/tcp abiertos y estimó que el sistema remoto pertenecía a la familia Linux. La detección del sistema operativo es aproximada y se basa en la respuesta de la pila TCP/IP.

7. Análisis de tráfico con Wireshark

7.1 Tráfico ICMP

Wireshark se ejecutó sobre la interfaz eth0 de Kali. Durante una prueba de ping se observaron paquetes ICMP Echo Request desde 10.10.10.10 hacia 10.10.10.20 y las correspondientes respuestas Echo Reply desde Debian hacia Kali.

Para generar el tráfico ICMP que posteriormente se analizó en Wireshark, desde Kali Linux se ejecutó una prueba de ping hacia el equipo Debian. Esta comunicación genera paquetes ICMP Echo Request y Echo Reply, que luego pueden observarse en la captura realizada con Wireshark y permiten verificar el intercambio de tráfico entre ambos equipos de la red local simulada.

```

(kali@kali)-[~]
$ ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=2.57 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.029 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.034 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.037 ms

— 10.10.10.10 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3032ms
rtt min/avg/max/mdev = 0.029/0.667/2.570/1.098 ms

```

Figura 8. Prueba de ping realizada desde Kali Linux hacia Debian para generar tráfico ICMP.

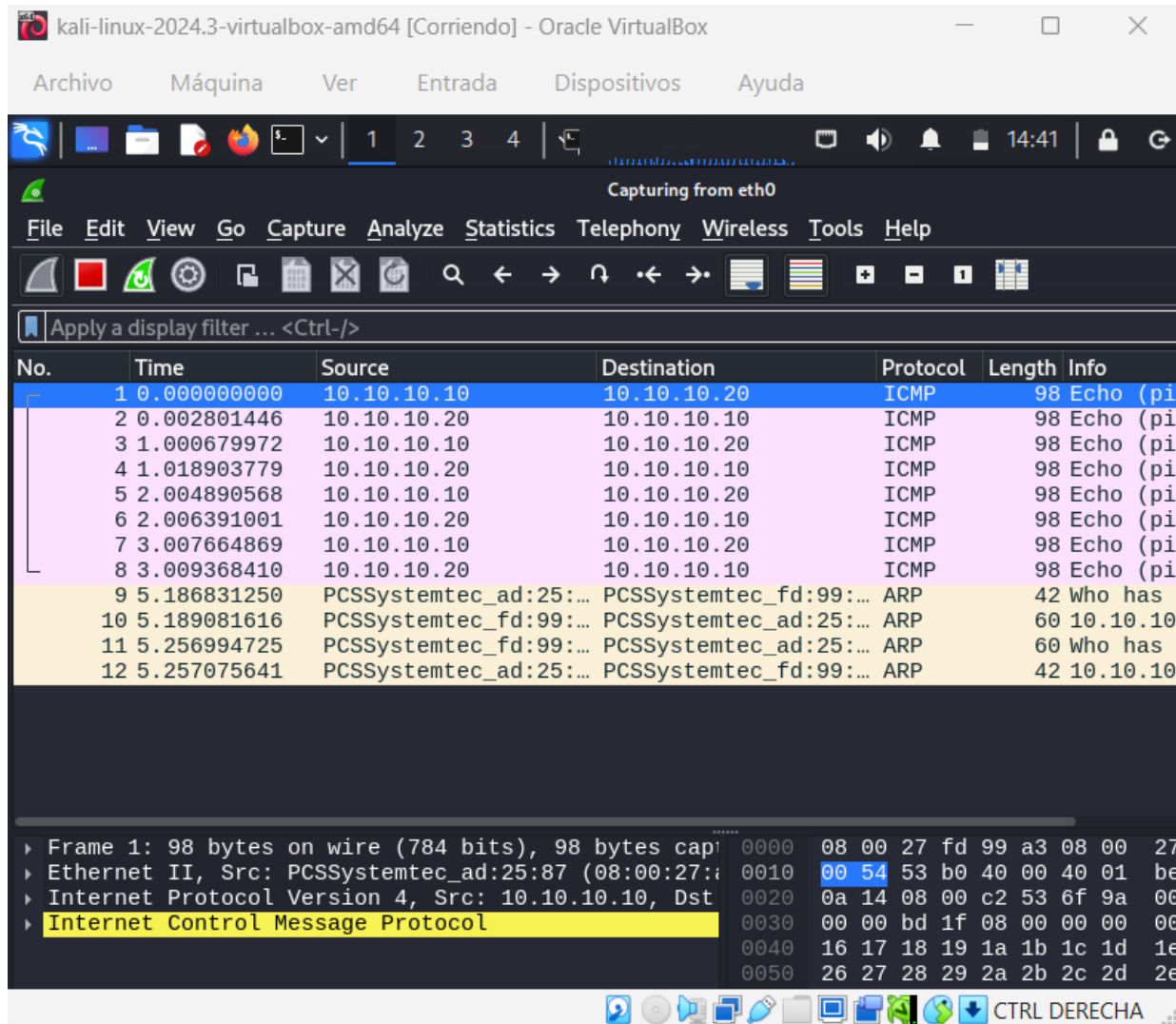


Figura 9. Captura ICMP entre Kali y Debian mediante Wireshark.

7.2 Tráfico HTTP y establecimiento TCP

Durante una solicitud HTTP hacia Apache se observó el establecimiento de la sesión TCP y el intercambio de datos. La captura muestra la secuencia SYN, SYN-ACK y ACK correspondiente al three-way handshake, seguida de la solicitud GET / HTTP/1.1 y la respuesta HTTP/1.1 200 OK. Finalmente aparecen paquetes FIN/ACK para cerrar la conexión.

Para provocar la consulta HTTP capturada en Wireshark, desde Kali Linux se ejecutó el comando `curl http://10.10.10.20`. El comando solicitó la página alojada en Apache y mostró en consola el contenido HTML recibido desde Debian.

```
(kali@kali)-[~]
$ curl http://10.10.10.20

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3
.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
  <title>Apache2 Debian Default Page: It works</title>
  <style type="text/css" media="screen">
    * {
      margin: 0px 0px 0px 0px;
      padding: 0px 0px 0px 0px;
    }

    body, html {
      padding: 3px 3px 3px 3px;

      background-color: #D8DBE2;

      font-family: Verdana, sans-serif;
      font-size: 11pt;
      text-align: center;
    }

    div.main_page {
      position: relative;

```

Figura 10. Solicitud HTTP realizada con curl desde Kali hacia el servidor Apache de Debian.

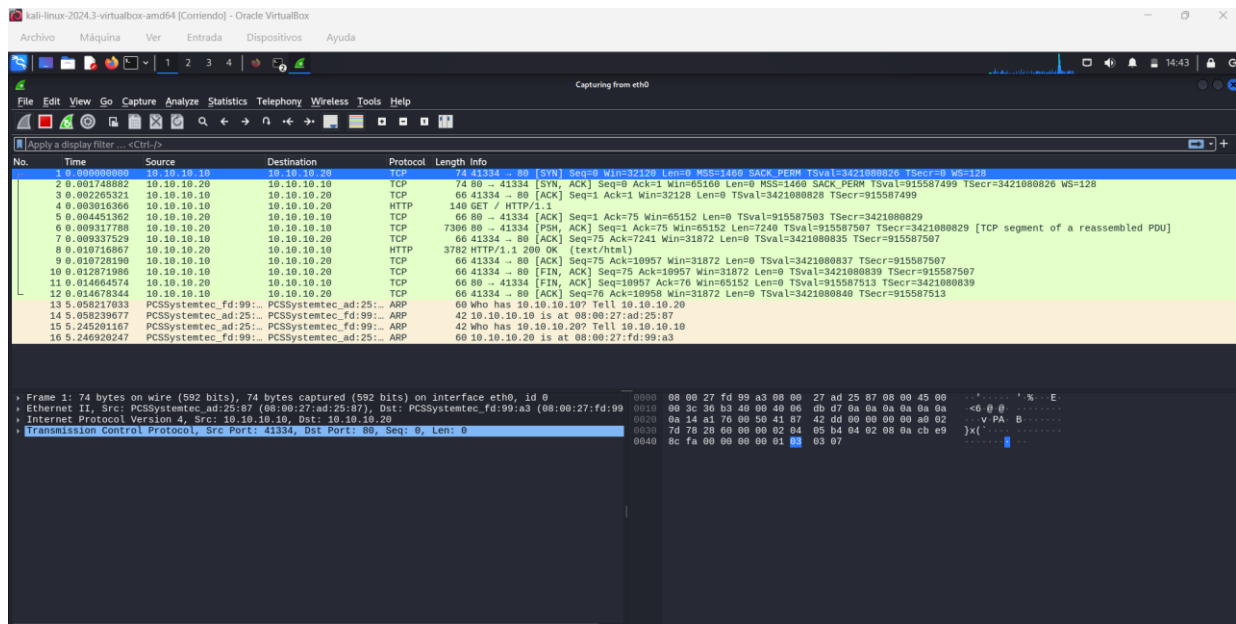


Figura 11. Captura del three-way handshake TCP y comunicación HTTP entre Kali y Debian.

8. Prueba intermedia de firewall

Para acercar la práctica al concepto de filtrado planteado por la actividad original, se implementó una regla de firewall directamente sobre Debian mediante iptables. El objetivo fue bloquear únicamente el tráfico HTTP proveniente de Kali, manteniendo operativa la conectividad ICMP.

La regla aplicada fue:

```
sudo iptables -I INPUT 1 -p tcp --dport 80 -s 10.10.10.10 -j DROP
```

La regla indica que todo paquete TCP recibido desde 10.10.10.10 cuyo destino sea el puerto 80 debe descartarse.

```

renzo@Debian:~$ sudo iptables -I INPUT 1 -p tcp --dport 80 -s 10.10.10.10 -j DROP
[sudo] password for renzo:
renzo@Debian:~$ sudo iptables -L INPUT -n --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination              tcp dpt:80
1 DROP          tcp  --  10.10.10.10             0.0.0.0/0                 tcp dpt:80
renzo@Debian:~$ _

```

Figura 12. Regla iptables activa bloqueando HTTP desde Kali hacia Debian.

8.1 Resultado con el firewall activo

Con la regla activa, curl dejó de conectarse al puerto 80 y finalizó por timeout. Nmap pasó a identificar el puerto 80 como “filtered”, mientras que el ping continuó funcionando con 0 % de pérdida. Esto demuestra que el bloqueo afectó únicamente al servicio HTTP y no a toda la comunicación de red.

```

(kali㉿kali)-[~]
└─$ curl --connect-timeout 5 http://10.10.10.20
curl: (28) Failed to connect to 10.10.10.20 port 80 after 5005 ms: Timeout was reached

(kali㉿kali)-[~]
└─$ sudo nmap -p 80 10.10.10.20
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-06 15:02 EDT
Nmap scan report for 10.10.10.20
Host is up (0.0017s latency).

PORT      STATE      SERVICE
80/tcp    filtered  http
MAC Address: 08:00:27:FD:99:A3 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 16.93 seconds

(kali㉿kali)-[~]
└─$ ping -c 4 10.10.10.20
PING 10.10.10.20 (10.10.10.20) 56(84) bytes of data.
64 bytes from 10.10.10.20: icmp_seq=1 ttl=64 time=1.81 ms
64 bytes from 10.10.10.20: icmp_seq=2 ttl=64 time=0.845 ms
64 bytes from 10.10.10.20: icmp_seq=3 ttl=64 time=1.61 ms
64 bytes from 10.10.10.20: icmp_seq=4 ttl=64 time=1.17 ms

— 10.10.10.20 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.845/1.357/1.809/0.375 ms

(kali㉿kali)-[~]
└─$

```

Figura 13. HTTP bloqueado: timeout en curl, puerto 80 filtered en Nmap y ping aún operativo.

8.2 Restauración del servicio

Una vez finalizada la prueba, se eliminó la regla de iptables. La solicitud HTTP volvió a responder con 200 OK, confirmando que el bloqueo anterior era consecuencia directa de la regla aplicada y no de una falla de Apache o de la red.

```
(kali@kali)-[~]
$ curl -I http://10.10.10.20
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 13:28:19 GMT
Server: Apache/2.4.67 (Debian)
Last-Modified: Wed, 02 Sep 2026 12:37:28 GMT
ETag: "29cd-65a7f4c559179"
Accept-Ranges: bytes
Content-Length: 10701
Vary: Accept-Encoding
Content-Type: text/html

(kali@kali)-[~]
$
```

Figura 14. Acceso HTTP restaurado después de eliminar la regla de firewall.

9. Relación con la actividad original

La actividad INFRA-10 original propone una arquitectura más completa con pfSense, una LAN interna, una DMZ y una WAN simulada. En este trabajo se optó por una versión reducida para concentrarse en los conceptos fundamentales y obtener evidencia práctica con menos componentes.

Concepto de INFRA-10	Implementación realizada	Estado
Red virtual aislada	Red interna labLAN en VirtualBox	Realizado
Cliente de pruebas	Kali Linux	Realizado
Servidor de prueba	Debian con SSH y Apache	Realizado
Nmap	Escaneo de puertos, versiones y SO	Realizado
Wireshark	Capturas ICMP y HTTP/TCP	Realizado
Firewall / segmentación	Regla iptables sobre Debian; no se implementó pfSense/DMZ/WAN	Parcial / simplificado

10. Evidencias obtenidas

- Configuración IPv4 de Kali Linux.
- Configuración IPv4 de Debian y conectividad mediante ping.
- Estado del servicio Apache.
- Respuesta HTTP 200 OK desde Kali.
- Acceso gráfico desde Kali a la página predeterminada de Apache2.
- Escaneo Nmap de servicios y versiones.
- Escaneo Nmap combinado con detección de sistema operativo.
- Captura Wireshark de ICMP Echo Request / Echo Reply.
- Prueba de ping desde Kali Linux hacia Debian utilizada para generar el tráfico ICMP analizado en Wireshark.
- Captura Wireshark del three-way handshake y HTTP GET / 200 OK.
- Solicitud curl utilizada para generar el tráfico HTTP analizado en Wireshark.
- Regla de firewall aplicada mediante iptables.
- Bloqueo del puerto 80 comprobado mediante curl y Nmap.
- Restauración del acceso HTTP tras retirar la regla.

11. Conclusión

El laboratorio permitió construir y validar una red local virtual aislada utilizando Kali Linux y Debian. Se comprobó conectividad entre ambos equipos, se levantaron y detectaron servicios de red, se analizaron paquetes ICMP y HTTP/TCP con Wireshark y se verificó el comportamiento de una regla de firewall mediante iptables.

Aunque la implementación no reproduce la topología completa con pfSense, DMZ y WAN simulada de la consigna original, conserva los principales conceptos técnicos del ejercicio: direccionamiento IP, comunicación entre hosts, exposición y descubrimiento de servicios, monitoreo de tráfico y control de acceso mediante reglas de filtrado.

La prueba de firewall resultó especialmente útil para demostrar de forma controlada la diferencia entre conectividad general y acceso a un servicio específico: el host Debian continuó respondiendo ICMP mientras el puerto HTTP permanecía filtrado, y volvió a estar disponible al retirar la regla.

12. Documento de referencia

INFRA-10 · Laboratorio Paso a Paso: Crear, Probar y Explicar un Entorno Simulado. Actividad base proporcionada para el curso.